# 📱 Acessando Recursos Nativos e Hardware com Flutter

## 1. Introdução

- Até o momento estávamos em uma bolha chamada Dart.
- O Flutter é excelente para desenhar telas, gerenciar estados e manipular dados.
- Mas o Flutter sozinho é cego, surdo e mudo para o mundo físico. Ele não conhece a lente da câmera, não sabe como ligar a lanterna, nem ler o GPS...
- Hoje vamos atravessar essa bolha e interagir com o hardware do celular.

## 2. Plugin vs Pacote

**Pacote (Package):**

- Código Dart puro. Não conversa com o hardware.
- Apenas resolve lógica e estrutura de dados.
- *Exemplos:* `mobx`, `dio`, `intl`.

**Plugin:**

- É um pacote com superpoderes.
- Contém código nativo em pastas específicas (Kotlin/Java para Android, Swift/Objective-C para iOS).
- *Exemplos:* `image_picker`, `geolocator`, `flutter_blue_plus`.

**E se não existir plugin para o que eu preciso?**
Usamos **Platform Channels** para criar nossa própria ponte de comunicação com o código nativo:

- **MethodChannel:** Usado para chamadas pontuais (uma pergunta e uma resposta). Ex: "Qual é o nível da bateria agora?".
- **EventChannel:** Usado para escutar um fluxo contínuo de dados. Ex: "Me avise sempre que a porcentagem da bateria mudar".

```
void main() async {
    // Exemplo de MethodChannel (Chamada única)
    final methodChannel = MethodChannel('br.com.bfeiten.native_playground/authenticatio
    final methodResult = await methodChannel.invokeMethod('check-biometric');

    // Exemplo de EventChannel (Ouvindo um fluxo contínuo)
    final eventChannel = EventChannel('br.com.bfeiten.native_playground/authentication'
    eventChannel.receiveBroadcastStream().listen((event) {
        print('Novo evento recebido do nativo: $event');
    });
}
```

# 3. Implementações Práticas (O Valor de Negócio)

### Câmera e Galeria ( `image_picker` )

- Imaginem o estoquista de uma loja ou um operador no campo. Ele encontrou uma caixa com avaria ou um equipamento quebrado. Pedir para esse cara, no meio da correria, com a mão suja ou debaixo de sol, digitar um relatório de três parágrafos no teclado do celular é pedir para ele odiar o seu aplicativo. Qual é a mágica do `image_picker` ? Você dá o poder da visão para o app. Em vez de digitar, ele saca o celular, bate a foto da avaria, e o sistema já salva no servidor. Uma foto elimina cinco minutos de digitação e resolve o problema de comunicação na hora. Nós usamos a câmera para poupar o tempo do usuário.

### Feedback Tátil / Vibração ( `haptic_feedback` / `vibration` )

- Pensem no usuário do nosso aplicativo que está na linha de frente fazendo uma tarefa repetitiva: conferindo uma lista de produtos, aprovando pedidos ou calibrando um sensor. Ele está prestando atenção no mundo físico ao redor dele, e não olhando fixamente para a tela do celular. Se ele aperta em 'Salvar' e o servidor recusa, não adianta você colocar uma mensagem vermelha linda na tela, porque **ele não está olhando**. Mas se o celular der um sinal, um "bzz-bzz" agressivo na mão dele, o cérebro dispara um alerta na hora. O feedback tátil permite que o aplicativo converse com o usuário através do tato, garantindo que ele perceba um erro ou um sucesso mesmo estando completamente desatento à interface visual.

### Abertura de Links ( `url_launcher` )

- Visualizem a jornada de suporte ou de vendas. O seu cliente está usando o app, travou em alguma tela e precisa de ajuda. Se você colocar um texto escrito: "Dúvidas? Adicione o número (51) 99999-9999 e mande um WhatsApp", o que acontece? O usuário tem que decorar o

número, sair do seu app, abrir a agenda, criar um contato, abrir o WhatsApp, procurar a pessoa e digitar "Oi". Ele desiste no segundo passo. Com o `url_launcher`, a gente cria um botão verde brilhante: "Falar com o Suporte". O usuário dá um único clique, o sistema operacional abre as portas, e ele cai magicamente dentro do WhatsApp com a mensagem "Olá, preciso de ajuda na tela X" já digitada. Você não perde o usuário, você constrói uma ponte VIP para ele.

# 4. Catálogo de Possibilidades (Outras Implementações)

- `torch_light` : Controlar a lanterna do celular.
- `battery_plus` : Verificar nível de bateria e modo de economia de energia.
- `share_plus` : Compartilhar nativamente textos, links, arquivos, fotos, vídeos, PDFs, etc.
- `video_player` : Execução de vídeos com controle facilitado.
- `local_auth` : Verificação biométrica (Face ID, Touch ID, Íris) e checagem de hardware.
- `sensors_plus` :
  - **Acelerômetro:** Mede forças e gravidade (Contar passos, rotação de tela, chacoalhar).
  - **Barômetro:** Mede pressão atmosférica (Cálculo de altitude, auxílio ao GPS eixo Z).
  - **Giroscópio:** Mede rotação angular (Essencial para AR/VR, estabilização de panorâmicas).
  - **Magnetômetro:** Mede campos magnéticos (Bússolas e detecção de metais).
- `connectivity_plus` : Status de conexão (Wi-Fi, Dados Móveis, VPN, etc).
- `flutter_tts` : Text-to-Speech (faz o celular "falar" um texto).

# 4. Mais Opções de Implementação

`torch_light` : Controlar a lanterna do celular.

`battery_plus`

- Verificar e acompanhar o nível de bateria do celular.
- Verificar se o dispositivo está em modo de "Economia de energia".

`share_plus` : Compartilhar informações nativamente pelo app (textos, links, arquivos, fotos, vídeos, PDFs, etc).

`video_player` : Permite a execução de vídeos em um player com controle facilitado.

`local_auth`

- Permite solicitar a verificação de segurança biométrica (Face ID, Touch ID, leitura de íris).
- Verifica se o dispositivo possui hardware e configuração para biometria.

`sensors_plus`
Permite acompanhar os dados físicos do dispositivo.

- **Acelerômetro:** Mede as forças (incluindo a gravidade a 9,81m/s²) atuando no aparelho.
  - *Uso:* Contar passos identificando picos de movimento, detectar rotação de tela ou o gesto de chacoalhar o celular.
- **Barômetro:** Mede a pressão atmosférica.
  - *Uso:* Calcular altitude, acelerar o tempo de "lock" do GPS fornecendo o eixo Z (altura) e auxiliar na previsão do tempo baseada em mudanças bruscas de pressão.
- **Giroscópio:** Mede a taxa de rotação angular do celular (em torno do próprio eixo).
  - *Uso:* Essencial para Realidade Aumentada (AR) e Virtual (VR) para saber onde a câmera aponta de forma fluida. Também usado em estabilização de fotos panorâmicas e como alternativa de navegação quando o GPS falha (como em túneis).
- **Magnetômetro:** Mede campos magnéticos.
  - *Uso:* Funcionamento de bússolas (apontando para o norte verdadeiro nos mapas) e detecção de objetos metálicos próximos por distorção magnética.

`connectivity_plus` : Permite verificar o status de conexão com a internet e identificar o tipo de rede em uso (Wi-Fi, Dados Móveis, Ethernet, Bluetooth, VPN, etc).

`flutter_tts` (Text-to-Speech): Usa as configurações de acessibilidade de voz do próprio celular para o sistema ler em voz alta um texto (falar).

# 🛠️ Guia Técnico de Implementação

Agora que entendemos a teoria, vamos ao **como** isso foi construído neste projeto.

## 📂 Mapeamento de Funcionalidades

| Recurso | Pacote Utilizado | Localização no Código | O que observar |
|---|---|---|---|
| **Câmera e Galeria** | `image_picker` | `lib/features/camera_demo/` | Como o `CameraAdapter` isola a lógica do pacote. |

| Recurso | Pacote Utilizado | Localização no Código | O que observar |
|---|---|---|---|
| **Feedback Tátil** | `vibration` / `haptic_feedback` | `lib/features/haptic_demo/` | A diferença entre vibrações simples e padrões hapticos. |
| **Links Externos** | `url_launcher` | `lib/features/url_launcher_demo/` | Como formatar URLs para WhatsApp e Web. |

## 🔍 Detalhes de Arquitetura

**1. Camada de Adaptação (Infrastructure)**
Seguindo a arquitetura do projeto, não chamamos os pacotes nativos diretamente nas páginas.

- **Por que?** Se amanhã mudarmos o pacote `image_picker` por outro, alteramos apenas o `adapter`, e não todas as telas do app. Isso mantém o código sustentável.
- **Onde olhar:** Verifique os arquivos `*.adapter.dart`.

**2. Configurações Nativas (Permissões)**
Recursos nativos exigem "permissão" do sistema operacional. Se o app fechar ou a função não disparar, verifique:

- 🤖 **Android:** Arquivo `android/app/src/main/AndroidManifest.xml` → Procure por tags `<uses-permission>`.
- 🍎 **iOS:** Arquivo `ios/Runner/Info.plist` → Procure por chaves de "Usage Description" (ex: `NSCameraUsageDescription`). O iOS exige que você explique ao usuário **por que** precisa da permissão.

# 🚀 Desafio Final

Tente implementar um dos seguintes recursos extras utilizando a mesma estrutura de **Adapter** → **Store** → **Page**:

1. **Lanterna:** Utilize o pacote `torch_light` para criar um botão de ligar/desligar a lanterna.
2. **Nível de Bateria:** Utilize o `battery_plus` para exibir a porcentagem da bateria na Home.
3. **Compartilhamento:** Utilize o `share_plus` para compartilhar um texto fixo ou a URL do app.

**Dica de Ouro:**

1. Adicione a dependência no `pubspec.yaml`.
2. Pense em quais dados o plugin te fornece e quais você realmente vai precisar na tela (liste isso para si).
3. Crie o `Adapter` para a funcionalidade.
4. Crie a `Store` para gerenciar o estado.
5. Implemente a `Page` para a interface.